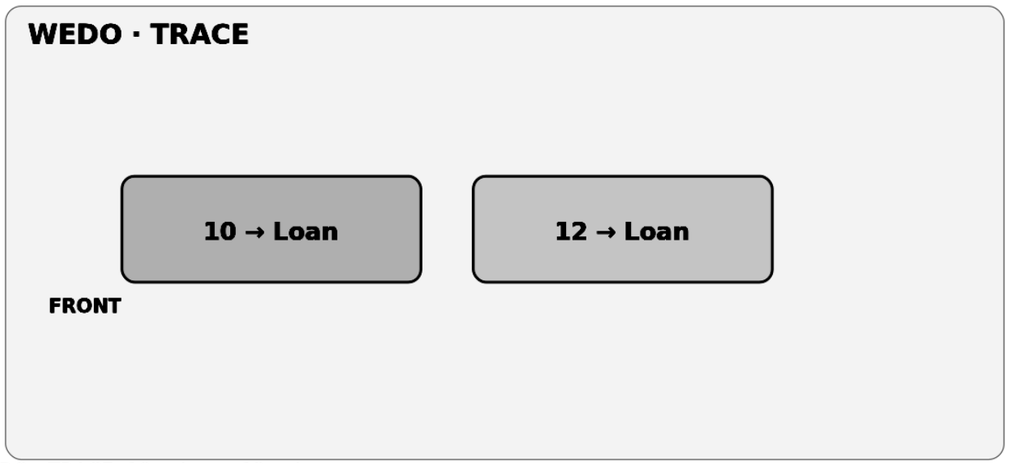
# We Do source CHECK · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

**A different library uses Dictionary<int,Loan> keyed by ID, entries 10 and 12. Which ID does lookup 12 retrieve**

A dictionary retrieves by key; it does not enforce FIFO service order.




WEDO · TRACE: 10 → Loan; 12 → Loan





**DO NOW — RECALL**

# Lesson 057 · C# Data Structures Mini-project: Library Lo

Name: _____ Class: _____ Date: _____

**1. What does a constructor initialise?**

**2. What does Queue<T>.Dequeue do?**

**3. Why choose a unique ID for a record?**

**4. A zero-indexed C# int[,] array a has 3 rows and 2 columns. Nested loops visit r=0 to 2 and c=0 to 1 inclusive, setting a[r,c]=10*r+c. Determine the final values in the last row, from left to right.**

DO NOW — CHECK / OBJECTIVES

# Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

**1. Model answer**

New object's fields/properties.

**2. Model answer**

Removes/returns oldest enqueued item.

**3. Model answer**

Names/labels can repeat; ID enables unambiguous lookup.

**4. Model answer**

20, 21. The last row has index 2, so its values are 10*2+0 and 10*2+1.

**By the end, you can…**

• Class Loan models a single loan request, containing a student and book title initialised in its constructor.

• Queue<Loan> stores objects in requested service order. Enqueue adds a new request and Dequeue retrieves oldest.

• For a more complete system, use a Dictionary keyed by unique loan ID, validation and tests; do not treat a name as necessarily unique.

• Introduce OCR file reading/writing so loan records can persist beyond the current in-memory queue.

**Vocabulary — know these terms**

model — Representation of a real-world concept.

constructor — Initialises new object.

Queue<T> — Generic FIFO collection.

validation — Checks input meets rules.

openWrite — Open an OCR pseudocode file for writing, replacing previous contents.

openRead — Open an OCR pseudocode file for reading stored lines.





WE DO — CHECK AND IMPROVE

# Source-grounded model answers

CHECK only after completing the guided practice and Exit Ticket. Use GREEN PEN to improve your method and explain any error.

**Guided 1 — CHECK**

Explain
A dictionary retrieves by key; it does not enforce FIFO service order.

**Guided 2 — CHECK**

Predict what is printed
Identify the role of endOfFile() and the effect of each readLine() call.
Follow two reads
The first call prints Ada,Holes ; the second prints Ben,Dune . The file position advances after each read.
Stop safely
Once no unread lines remain, endOfFile() is true, the loop stops, and f.close() releases the reader. This example reads text; it does not reconstruct Loan objects automatically.

EXIT TICKET / REFLECTION

# Check your understanding

Complete the actual lesson Exit Ticket before checking.

**A pupil's name is always guaranteed unique, so it is a perfect database key.**

**Circle: Confident / Getting there / Need help**

Explain what you can now do independently.

**GREEN PEN — Correct and explain**

Choose one error. Write the correct method and explain precisely why your first response was wrong.





NEW LEARNING

# Read, annotate and explain

**Understand**

Class Loan models a single loan request, containing a student and book title initialised in its constructor.

**Model**

Queue<Loan> stores objects in requested service order. Enqueue adds a new request and Dequeue retrieves oldest.

**Evaluate**

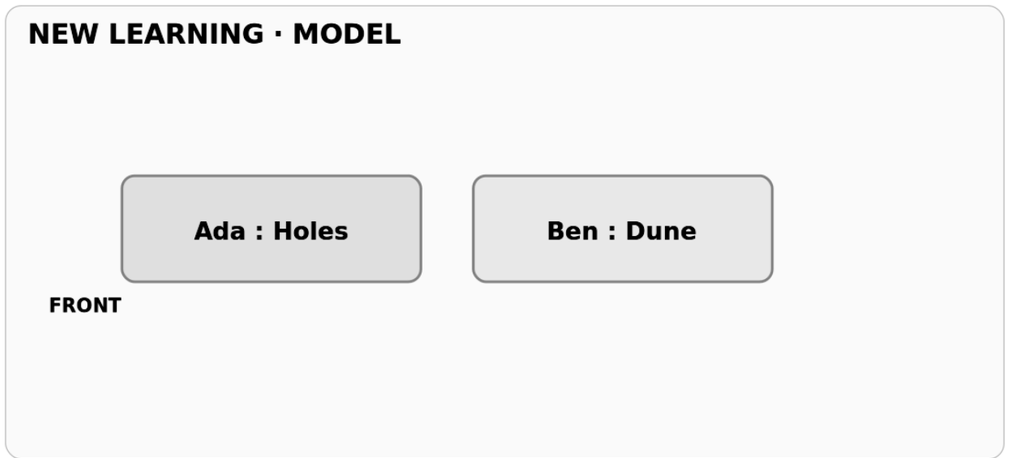
For a more complete system, use a Dictionary keyed by unique loan ID, validation and tests; do not treat a name as necessarily unique. A queue exists only while the program keeps it in memory; a text file can persist loan records. In OCR examination pseudocode, openWrite plus writeLine writes a file (overwriting existing contents), while openRead plus readLine retrieves lines. Always close() the file.

**Explain one step in your own words, using the correct technical terms.**

# New Learning · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

### New Learning




NEW LEARNING · MODEL: Ada : Holes; Ben : Dune





**YOU DO — INDEPENDENT PRACTICE**

# Questions · Part 2

Answer independently. Show code, calculations or a justified explanation.

### Question 3

Outline how to prevent duplicate loan IDs and reject empty student names; then name the OCR file operations for writing one saved loan and reading a line back. [8]

YOU DO — INDEPENDENT PRACTICE

# Questions · Part 1

Answer independently. Show code, calculations or a justified explanation.

### Question 1

Explain two reasons Loan is a class rather than separate parallel arrays for pupil and book. [4]

### Question 2

Write C# creating a Queue<Loan>, enqueue two instances, then safely dequeue one using Count. [8]





I DO — WORKED EXAMPLE

# Follow the worked method

### Trace two loan objects in a queue: Ida first, Tom second. Which is processed first?

### Explain

Queue is FIFO, so the first enqueued request is processed first.

### Why a file?

Unlike the queue, the text file can hold a record across program runs. openWrite creates a write stream and replaces any existing contents of the target file; it is not an append operation.

### Trace the file contents

First writeLine stores Ada,Holes , then the second stores Ben,Dune . They become two separate text lines. This is a simple illustrative format, not a robust CSV parser.

### Explain the effect

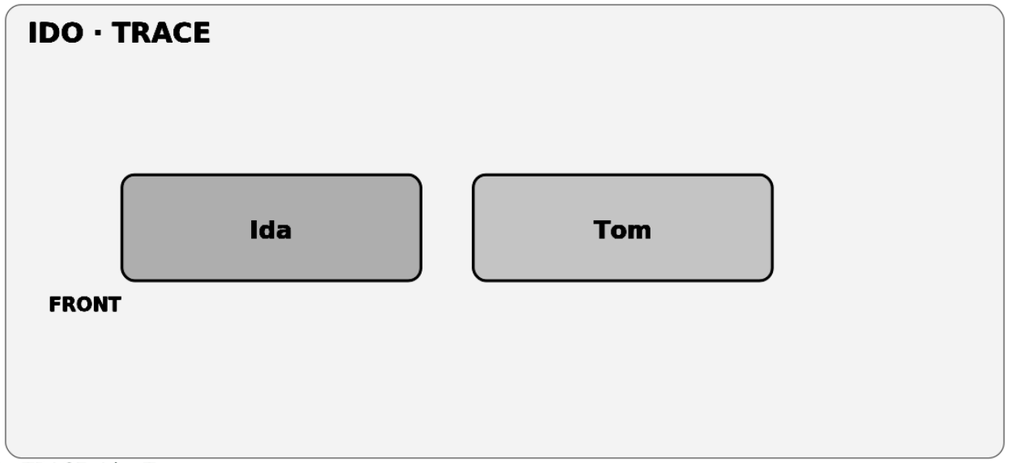
f.close() finishes the write operation and releases the file. Do not confuse storing records in a queue or dictionary with saving them permanently.

# I Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

**Trace two loan objects in a queue: Ida first, Tom second. Which is processed first?**

Queue is FIFO, so the first enqueued request is processed first.




IDO · TRACE: Ida; Tom





**WE DO — GUIDED PRACTICE**

# Try → Discuss → Improve

TRY: Attempt the guided questions in BLACK PEN. DISCUSS: Compare approaches with a partner, explain your reasoning and agree one improvement. IMPROVE: After the Exit Ticket, use the source CHECK model in GREEN PEN to correct and explain errors.

### Guided 1 — TRY

A different library uses Dictionary<int,Loan> keyed by ID, entries 10 and 12. Which ID does lookup 12 retrieve?
Source code / input:
loansById[12] returns the Loan stored under ID 12, regardless of insertion order.

### Guided 2 — TRY

Read loans.txt , containing Ada,Holes and Ben,Dune on separate lines.
Source code / input:

```
// OCR H446 examination pseudocode
f = openRead("loans.txt")
while NOT f.endOfFile()
print(f.readLine())
endwhile
f.close()
```